BayesInference Multi-Language Conversion Guide

This guide provides a reference for converting BayesInference (BI) code between Python, R (BIR), and Julia (BIJ).

Language Comparison Table

Feature	BI Python	BI R (BIR)	BI Julia (BIJ)
Import	`from BI import bi`	`m = importBI()`	`m = importBI()`
Initialize	`m = bi(platform='cpu')`	`m = importBI(platform="cpu")`	`m = importBI(platform="cpu")`
Model	`def model(var1, var2):`	`model <- function(var1, var2) {`	`@BI function model(var1, var2)`
Priors	`m.dist.normal(0, 1, name='x')`	`bi.dist.normal(0, 1, name='x')`	`m.dist.normal(0, 1, name="x")`
Likelihood	`m.dist.normal(mu, sigma, obs=y)`	`bi.dist.normal(mu, sigma, obs=y)`	`m.dist.normal(mu, sigma, obs=y)`
Link (Logit)	`jax.nn.sigmoid(x)`	`jax$nn$sigmoid(x)`	`m.link.inv_logit(x)`
Exponential	`jnp.exp(x)`	`jnp$exp(x)`	`jnp.exp(x)`
Logarithm	`jnp.log(x)`	`jnp$log(x)`	`jnp.log(x)`
Indices	`x[idx]`	`x[idx]`	`x[idx]`
Fitting	`m.fit(model)`	`m$fit(model)`	`m.fit(model)`
Summary	`m.summary()`	`m$summary()`	`m.summary()`

Examples

1. Linear Regression

[BI Python]

```python
def model(x, y):
    alpha = m.dist.normal(0, 10, name='alpha')
    beta = m.dist.normal(0, 1, name='beta')
    sigma = m.dist.exponential(1, name='sigma')
    mu = alpha + beta * x
    m.dist.normal(mu, sigma, obs=y)
```

[BI R (BIR)]

```r
model <- function(x, y) {
    alpha = bi.dist.normal(0, 10, name='alpha')
    beta = bi.dist.normal(0, 1, name='beta')
    sigma = bi.dist.exponential(1, name='sigma')
    mu = alpha + beta * x
    bi.dist.normal(mu, sigma, obs=y)
}
```

[BI Julia (BIJ)]

```julia
@BI function model(x, y)
    alpha = m.dist.normal(0, 10, name="alpha")
    beta = m.dist.normal(0, 1, name="beta")
    sigma = m.dist.exponential(1, name="sigma")
    mu = alpha + beta * x
    m.dist.normal(mu, sigma, obs=y)
end
```

Tips for Conversion

1. **Object Scope**:
    - In **Python**, `m` is the central object.
    - In **R**, `bi.dist.*` is used for distributions globally once BI is imported, while `m` is used for methods like `fit` and `summary`.
    - In **Julia**, `m` is used for both distributions (`m.dist`) and methods.
2. **Accessing Modules**:
    - R uses `$` to access Python module members (e.g., `jax$nn$sigmoid`).
    - Python and Julia use `.` (e.g., `jax.nn.sigmoid`).
3. **Indexing**: Note that while R and Julia usually use 1-based indexing, the underlying JAX arrays in BI often follow the data indexing passed to the model. Best practice is to ensure your indices start from 0 if they are used to index JAX arrays.
4. **R Integer Vectors**: In BI R, the `shape` parameter for distributions must be specified as an integer vector using the `L` suffix, for example: `shape = c(1L, 10L)`.
5. **Macros**: Julia uses the `@BI` macro to correctly wrap the model function for the Python backend.